

PW 3: Sub-program (functions)

Goal

- Understand the different elements constituting a sub-program.
- Know how to test a sub-program.
- Understand the parameters and local variables.
- Know how to write a sub-program.

Exercise 1: Anatomy of a sub-program

- Considering the following code, explain what it is:
 - Signature
 - the documentation chain or "docstring"
 - specification
 - statements
- Implantation. The following questions relate to the implementation (the code) of the various functions of the listing
 - Indicate the behavior of the *return* statement?
 - What is the return value of a subroutine that does not contain a *return*?
 - Explain the behavior of the *assert* statement
 - Explain and compare the following two calls:
 - `minimum(1, 2)`
 - `minimum(elt2=-3, elt1=-5)`
 - Explain the meaning of
 - `if __name__ == '__main__':`
- Run the program

```
minimum.py > ...
1  '''The minimum function and its test programs.'''
2
3  def minimum(elt1, elt2):
4      '''
5          Return the smaller of the two elements as a parameter.
6
7          : param elt1: the first element.
8          : type elt1: any
9          : param elt2: the second element.
10         : type elt2: any
11         : returns: the smaller of the two elements.
12         '''
13         if elt1 < elt2:
14             return elt1
15         else:
16             return elt2
17
18
19  def test_min():
20      '''Tests the minimum of integers.'''
21      assert 1 == minimum(1, 2)
22      assert -5 == minimum(elt2=-3, elt1=-5)
23
24  def test_min_chaines():
25      '''Test the minimum of two strings.'''
26      assert 'cat' == minimum('cat', 'horse')
27
28  if __name__ == '__main__':
29      test_min()
30      test_min_chaines()
31      print('successful tests')
```

Exercise 2:

Using the following program:

- Can we use the same parameter name for sp1 and sp2?
- When does the 'a' parameter of sp1 exist?
- How to understand the expression sp1 (a = a) in main?
- What is the output of this program?
- Run the program with <http://www.pythontutor.com>

```
1  '''Understand the scope of variables and local variables.'''
2
3  def sp1(a):
4      b = 5
5      return a + b
6
7  def sp2(a):
8      return 2 * a
9
10 def main():
11     print(sp1(2))
12     print(sp2(3))
13     a = 6
14     print(sp1(a=a))
15     print(b)
16
17 main()
18
```

Exercise 3:

Write a program that calculates the n^{th} power of a number using only sum and multiplication

- don't use `**`
- n and the number itself is given by the user.
- Don't forget to write a function that contains asserts to verify your code.
- "Docstring" is obligatory

Exercise 4:

A natural number is said to be perfect if it is equal to the sum of its divisors, excluding it. For example, 28 is a perfect number ($28 = 1 + 2 + 4 + 7 + 14$).

Two numbers N and M are said to be friends if the sum of the divisors of M (excluding M itself) is equal to N and the sum of the divisors of N (excluding N itself) is equal to M . For example, 220 and 284 are friends. Indeed, the sum of the divisors of 220 out of 220 is

$1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284$ and the sum of the divisors of 284 excluding 284 is $1 + 2 + 4 + 71 + 142 = 220$.

Our goal is to write two subprograms. The first displays in ascending order and progressively the perfect numbers from 2 to a given natural number. The second displays all the friendly numbers (N, M) between 2 and a given natural number MAX such as $0 < N < M \leq \text{MAX}$.

Before coding in Python, write the algorithm of the problem using the refining method.

Once done, you can write your code. DON'T FORGET THE "Docstring"